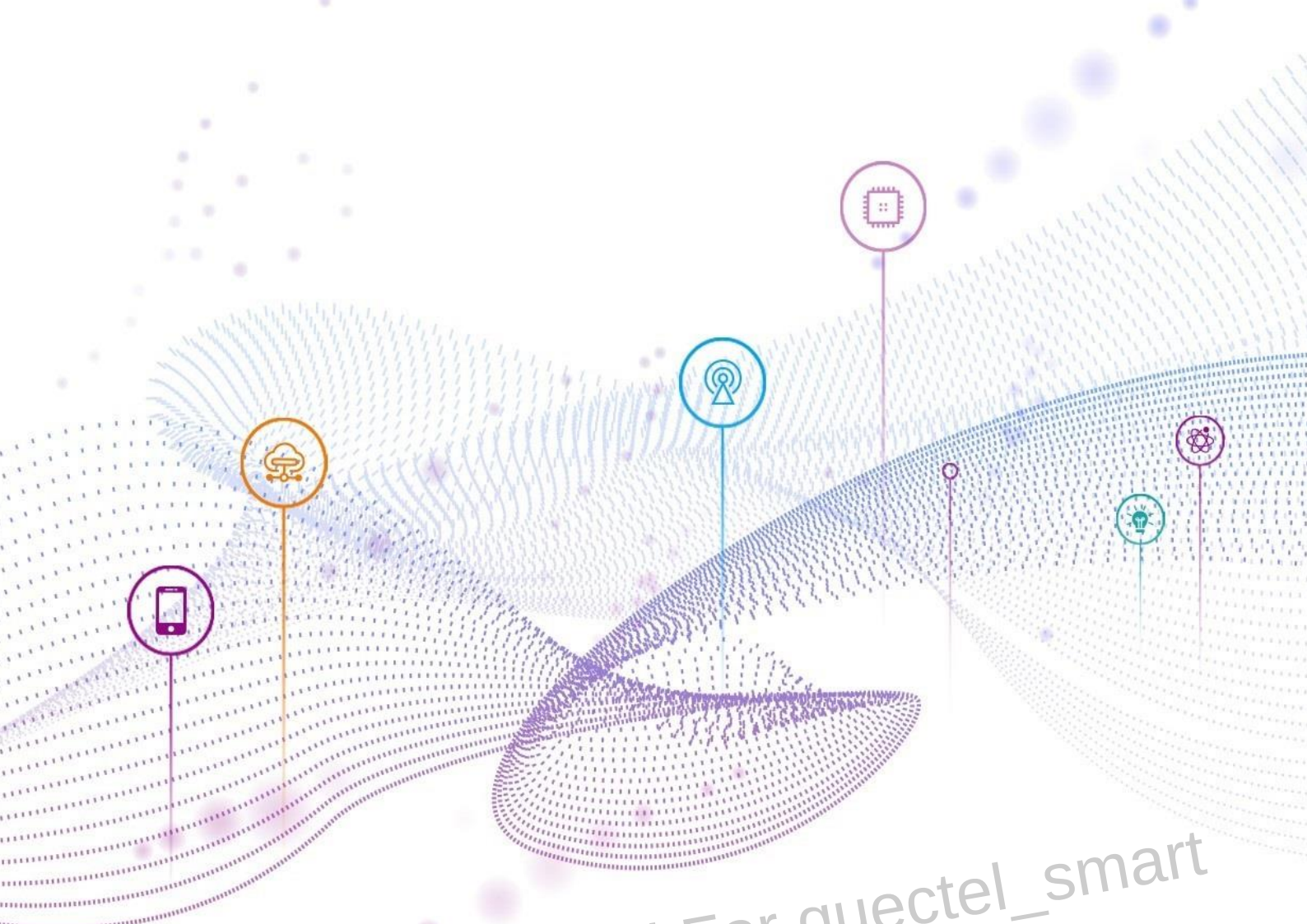




Unisoc Confidential For quectel_smart

Android10 SELinux 指导文档

文档版本
发布日期

V1.0
2020-05-19

版权所有 © 紫光展锐科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

Unisoc Confidential For quectel_smart

紫光展锐科技有限公司



前 言

概述

本文档详细地描述了 SELinux 模块基本内容，提供了 SELinux 权限的常见问题解决方法。

读者对象

本文档主要适用于所有研发人员阅读和参考。

缩略语

缩略语	英文全名	中文解释
SELinux	Security-Enhanced Linux	安全增强型 linux
AVC	Access Vector Cache	访问向量缓存

关键字

AVC, 新增访问限制, Selinux 常见问题

目 录

1 Selinux 基础知识	1
1.1 DAC 和 MAC	1
1.2 Selinux 状态	1
1.3 查看 Selinux 状态和开关 Selinux 限制	1
1.3.1 查看 selinux 状态	1
1.3.2 设置版本为 permissive 模式, 有三种方式	2
1.4 AVC 基础语法	2
1.5 修改验证方式	3
1.5.1 只修改 selinux 相关文件, 快速验证方式:	3
1.5.2 整体编译生成 pac 包到手机验证	3
1.6 SELinux 策略文件的位置	3
2 Selinux 常见问题	5
2.1 根据 log 中 avc 提示, 缺什么权限, 添什么权限	5
2.2 添加对 sysfs 的权限违反 google 策略	5
2.3 为文件配置安全上下文	6
2.3.1 为新增设备节点文件配置安全上下文并赋权限	6
2.3.2 为新增 proc 文件配置安全上下文并赋权限	6
2.4 为属性配置安全上下文	6
2.5 Android 属性 property 设置权限	7
2.5.1 新增属性, 首先需在 property_contexts 定义安全上下文	7
2.5.2 android 属性命名规范和 type 定义	7
2.5.3 新增 property 完整三部曲	8
2.6 te 文件有权限, 却因为 mls 规则导致访问失败	8
2.7 SELinux 安全上下文的恢复 (restorecon 命令)	9
2.8 使用 linux backtrace 打印出错函数堆栈信息	9
2.8.1 打印 avc backtrace 功能介绍	9
2.8.2 backtrace 使用方法	9

表目录

表 1-1 判断是否 selinux 权限问题	2
-------------------------------	---

Unisoc Confidential For quectel_smart

1 Selinux 基础知识

1.1 DAC 和 MAC

Linux 内核资源访问控制分为两类:

- **DAC 自主访问控制**
基于“用户-组-其他/读-写-执行”的权限检查, 进程理论上所拥有的权限与执行它的用户的权限相同, 该管理过于宽松, 如果获得 root 权限, 可以在 linux 系统干任何事情。
- **MAC 强制访问控制**
SELinux 是 MAC 机制的一种实现, 基于安全上下文和安全策略的安全机制, 用于补充 DAC 检查。访问系统资源时, 会先进行 DAC 检查, DAC 检查通过, 才能进行 MAC 检查, 如果 MAC 检查通过, 才能获得资源访问权。

1.2 Selinux 状态

selinux 分为三种模式:

(1)Disable

关闭模式, 不进行 selinux 权限检查。

(2)Permissive

宽容模式, 代表 SELinux 运作中, 违反 SELinux 规则只会有警告讯息(Avc denied),并不会实际限制 domain/type 的存取.这种模式可以作为 SELinux 的 debug 之用(看下什么原因导致无法访问)。

(3)Enforcing

强制模式, 代表 SELinux 运作中, 违反 SELinux 规则的行为将被阻止并记录到日志中。

1.3 查看 Selinux 状态和开关 Selinux 限制

1.3.1 查看 selinux 状态

通过命令行查看 selinux 状态:

adb shell getenforce

通过 avc log 来确认 selinux 状态:

avc log 结尾会有 permissive=1/0 的标示, 如果 permissive=1 说明是 Permissive 版本, 如果 permissive=0 说明是 Enforce 版本。

当代码执行出现异常时，可以在 **permissive** 版本上验证问题是否存在，来确定是否 **selinux** 权限问题。

表1-1 判断是否 selinux 权限问题

	Enforce 版本	Permissive 版本	结论
代码执行异常	存在	不存在	selinux 权限导致，结合 android 和 kernel log 的 avc 信息添加权限
代码执行异常	存在	存在	非 selinux 权限导致，需要排查其他原因

1.3.2 设置版本为 permissive 模式，有三种方式

方法一、命令行方式

```
adb root; adb shell setenforce 0
```

该方式仅适用于 userdebug 版本，系统重启修改会失效

方法二、修改 dts bootargs 参数

board 对应的 dts 文件里，在 bootargs 参数里增加 androidboot.selinux=permissive 字段。

该方式仅适用于 userdebug 版本，系统重启仍然有效。

方法三、修改 init 代码

可以修改 system/core/init/selinux.cpp 文件里的 IsEnforcing()

函数，将该函数直接返回 false。

```
bool IsEnforcing() {
```

```
+ return false ;
```

该方式适用 user 和 userdebug 版本，系统重启仍然有效。

1.4 AVC 基础语法

```
22:33:11.062 <36>[ 8.755284] c6 type=1400 audit(1325428390.909:11): avc: denied { read } for
pid=1834 comm= "gnss_download" name= "mmcblk0p17" dev= "tmpfs" ino=10268
scontext=u:r:gnss_download:s0 tcontext=u:object_r:block_device:s0 tclass=blk_file permissive=0
```

avc log 详解：

1. **avc: denied** 表示操作被否决，如果一些操作用规则 **auditallow** 设置，则它们产生授权的消息
2. **{ read }** 表示否决或授权的操作，**{ }**中含有实际尝试的操作；
3. **for pid=1834** 操作的源应用程序的进程 ID；

4. comm= “gnss_download” 表示被否决的应用程序，也就是主体进程名；
5. ino=10268 name= “mmcblk0p17” 操作尝试的目标文件或目录的路径，也就是客体资源名称；
6. dev=tmpfs 含有这个文件系统的设备节点。否决操作的客体在这个文件系统中；
7. ino=10268 目标文件或目录的节点号；
8. scontext=u:r:gnss_download:s0 被拒绝访问进程的安全上下文
9. tcontext=u:object_r:block_device:s0 即被拒绝访问的文件/目录的安全上下文；
10. tclass=blk_file 访问资源所属类别
11. permissive=0 当前是 enforcing 模式。
12. gnss_download (进程) 缺少对 block_device 类型 blk_file 文件 read 权限；

gnss_download.te 添加权限:

allow gnss_download block_device: blk_file {read};

1.5 修改验证方式

1.5.1 只修改 selinux 相关文件,快速验证方式:

(1) mma system/sepolicy 单独编译 sepolicy 模块验证.

(2)将编译产物 out/target/product/\${product}/路径下的 system/etc/selinux 和 vendor/etc/selinux 文件 push 到手机对应目录,重启手机验证修改是否生效。

具体方法:

```
adb push system/etc/selinux /system/etc/
```

```
adb push vendor/etc/selinux /vendor/etc/
```

1.5.2 整体编译生成 pac 包到手机验证

有条件的话最好 user & userdebug 本地编译仅仅能验证 userdebug 的编译情况。

android9.0 版本可以替换 system.image 和 vendor.image 到手机验证;android10.0 版本需先进行手机解锁.

1.6 SELinux 策略文件的位置

device/sprd/sharkle/common/sepolicy 和 system/sepolicy 路径下的 selinux 配置都生效.

(1)device/sprd/xxx/common/sepolicy 配置具体各项目 selinux 策略,我们修改 sepolicy 一般在这个路径.

(2)system/sepolicy google 原生策略,我们一般不在这个路径修改.

(3)/system/sepolicy/private/ system 分区 private

/system/sepolicy/public/ system 分区 public

/system/sepolicy/vendor/ vendor 分区

/system/sepolicy/prebuilts 版本兼容

(4)sepolicy is now divided into multiple portions:

public - policy exported on which non-platform policy developers may write additional policy. types and attributes are versioned and included in delivered non-platform policy, which is to be combined with platform policy.

private - platform-only policy required for platform functionality but which is not exported to vendor policy developers and as such may not be assumed to exist.

vendor - vendor-only policy required for vendor functionality. This policy can reference the public policy but cannot reference the private policy. This policy is for components which are produced from the core/non-vendor tree and placed into a vendor partition.

Unisoc Confidential For quectel_smart

2 Selinux 常见问题

2.1 根据 log 中 avc 提示，缺什么权限，添什么权限

avc: denied { read write } for pid=3483 comm="Binder_6" name="rtc0" dev="tmpfs" ino=4451
scontext=u:r:system_server:s0 tcontext=u:object_r:refnotify_device:s0 tclass=chr_file permissive=0

AVC 含义：

system_server（进程）缺少对 refnotify_device 类型 chr_file 文件 read write 权限；

参考公式：

device/sprd/xxx/common/sepolicy/scontext.te

+allow sccontext tcontext:tClass Permission;

添加权限：

device/sprd/xxx/common/sepolicy/system_server.te

+allow system_server refnotify_device:chr_file {read write};

可以设置 selinux 为 permissive，使所有相关 avc 暴露，添加权限。

2.2 添加对 sysfs 的权限违反 google 策略

例如，添加 allow system_app sysfs:file { write}; 违背如下 neverallow:

neverallow { appdomain -bluetooth -nfc }sysfs:dir_file_class_set write;

google 不允许应用进程写 sysfs 类型的文件，需要自定义 type，更改客体的安全上下文

解决办法：

(1) 参考原来的类型 sysfs 定义一种新 type，type 名字自定义

device/sprd/xxx /common/sepolicy/file.te

+type sysfs_xxx, fs_type,sysfs_type;

(2) 使用新定义的类型为文件配置安全上下文

/xxx /xxx (目标文件路径) u:object_r:sysfs_xxx:s0

(3) 赋予主体进程访问对应新类型文件的权限

common/sepolicy/system_app.te:

+allow system_app sysfs_xxx:file { write };

2.3 为文件配置安全上下文

2.3.1 为新增设备节点文件配置安全上下文并赋权限

第一步，已有类型种类很多，详情见 `system/sepolicy/public/device.te`

首先确认要添加的设备节点是否属于已有类型，如果已有定义，直接添加权限。如果没有，需要自定义类型 `type`。

```
device/sprd/xxx /common/sepolicy/device.te
```

```
+type autotest_device, dev_type;
```

第二步，配置安全上下文

```
device/sprd/xxx /Common/sepolicy/file_contexts
```

```
+ /dev/autotst u:object_r:autotest_device:s0
```

第三步，赋权限

```
device/sprd/xxx /Common/sepolicy/autotest.te
```

```
+allow autotest autotest_device:dir { search read };
```

2.3.2 为新增 proc 文件配置安全上下文并赋权限

`proc` 文件与特殊文件系统的安全上下文标记相关，比较常见的是 Bluetooth 会创建一个 `proc` 文件系统的文件

第一步，定义类型或者使用原有类型。

`proc_bluetooth_writable` 是已有类型，定义类型这步省略，如需重定义标签可以参照如下定义

```
system/sepolicy/public/file.te
```

```
type proc_bluetooth_writable, fs_type, proc_type;
```

第二步，配置安全上下文

```
device/sprd/xxx common/sepolicy/genfs_contexts
```

```
+ genfscon proc /bluetooth/sleep/btwrite u:object_r:proc_bluetooth_writable:s0
```

第三步，赋权限

```
device/sprd/xxx common/sepolicy/bluetooth.te
```

```
+ allow bluetooth proc_bluetooth_writable:file write;
```

2.4 为属性配置安全上下文

```
avc: denied { set } for property=persist.netmon.linger pid=1 uid=0 gid=0 scontext=u:r:vendor_init:s0  
tcontext=u:object_r:default_prop:s0 tclass=property_service permissive=0\x0a5.
```

以上 `avc` 为 `vendor_init` 设置 `persist.netmon.linger` 属性，缺少权限，`default_prop` 为新增属性的默认标签。直接添加：`allow vendor_init default_prop:property_service { set };`；违背 `neverallow`。

解决办法：

```
(1)device/sprd/xxx/ common/sepolicy/property.te
+type linger_prop, property_type;
(2)device/sprd/xxx/ common/sepolicy/property_contexts
+persist.netmon.linger    u:object_r: linger_prop:s0
(3)device/sprd/xxx / common/sepolicy/vendor_init.te
+allow vendor_init linger_prop:property_service { set };
```

2.5 Android 属性 property 设置权限

Android 属性 `property` 在系统中存放在一块共享内存中，每个进程都能读，但是只有 `init` 进程能够写，请求 `init` 进程帮助写属性值是需要 SELinux 权限的；

2.5.1 新增属性，首先需在 `property_contexts` 定义安全上下文

Google 有如下宏为对应进程设置属性的读写权限：

```
/system/sepolicy/public/te_macros
define(`set_prop', `
socket allow $1 property_socket:sock_file write; //因为这个 socket 属于 init 进程
allow $1 init:unix_stream_socket connectto; //赋予$1 进程有对$2 类型的属性有设置权限
allow $1 $2:property_service set;
`)
```

因为 `/dev/socket/property_service` `u:object_r:property_socket:s0`，设置属性都需要，通过 `property_service` 这个服务，这个服务对外接口是一个 `property_socket` 类型。通过 `allow` 语句添加 `property_service` `set` 权限，与使用 `set_prop` 宏，道理是一样的，建议使用 `set_prop` 宏为进程添加读写 `property` 的权限。

2.5.2 android 属性命名规范和 type 定义

(1)android 属性的名称是有一定的格式要求的，前缀必须用 `system\core\init\property_service.c` 中定义的前缀

vendor 属性名前缀：

`persist.vendor.**`

`vendor.**`

`ro.vendor.**`

属性名不包含 vendor,则为系统属性

(2)新增属性在 property.te 中添加属性的 type 定义

system 属性定义:

```
common/plat_sepolicy/public/property.te
```

```
type xxx_prop, property_type;
```

vendor 属性定义:

```
common/sepolicy/property.te
```

```
type xxx_prop, property_type;
```

2.5.3 新增 property 完整三部曲

根据属性的命名规范,选择定义在 system 分区和 vendor 分区

(1) 参照上面 property 定义,在对应路径下 property.te 中自定义一个类型;

(2) property_contexts 中对属性设定安全上下文;

系统属性:

```
device/sprd/xxx/common/plat_sepolicy/private/property_contexts
```

```
persist.ylog. u:object_r:ylog_prop:s0 //属性值后的.表示匹配以 persist.ylog 开头的属性
```

vendor 属性:

```
device/sprd/xxx/common/sepolicy/property_contexts
```

```
xxx.xxx u:object_r:xxx_prop:s0
```

(3) te 文件中赋予对应进程访问该类型属性的权利;

```
device/sprd/xxx/common/sepolicy/sprd_logmanager_app.te
```

```
set_prop(sprd_logmanager_app, ylog_prop)
```

2.6 te 文件有权限,却因为 mls 规则导致访问失败

avclog 信息中出现 "s0:c512,c768" 字眼,说明主体和客体安全级别不同

解决办法: 使主体进程关联 mltrustedsubject 或客体关联 mltrustedobject

例如:

```
avc log: scontext=u:r:untrusted_app_25:s0:c512,c768 tcontext=u:object_r:serial_device:s0。
```

untrusted_app_25 和 serial_device 安全级别不同, serial_device 关联 mltrustedobject, 穿越安全级别的限制。

```
device\sprd\dev**\common\sepolicy\file.te
```

```
+typeattribute serial_device mltrustedobject;
```

或 + typeattribute “主体进程” mltrustedsubject;

2.7 SELinux 安全上下文的恢复（restorecon 命令）

restorecon 命令用来恢复 SELinux 文件属性即恢复文件的安全上下文。

restorecon 命令格式如下：

```
[root@localhost ~] # restorecon [选项] 文件或目录
```

选项：

- -R：递归.当前目录和目录下所有的子文件同时恢复；

- -V：把恢复过程显示到屏幕上；

例如，依据 file_contexts 中的安全上下文配置，恢复标签

```
restorecon /mnt/vendor
```

(1)可首先在手机命令行执行如下命令验证是否能恢复成功，如果成功，可在 rc 文件中添加 Restorecon 命令恢复标签。

```
restorecon -R 文件路径
```

(2)如果在 rc 文件中添加 Restorecon 命令，不能恢复成功，抓 log 查看是否缺少相关权限，需添加相应权限。

2.8 使用 linux backtrace 打印出错函数堆栈信息

2.8.1 打印 avc backtrace 功能介绍

在解决 avc denied 权限问题时，可使用 backtrace 打印 avclog 对应出错函数堆栈信息,定位对应的代码逻辑。该功能为 debug feature，默认为关闭。可以设置 persist.vendor.avc.backtrace.enabled 属性为 1 来打开打印 avc backtrace 的功能。具体打印哪些进程的 avc backtrace，用户可以自定义。

2.8.2 backtrace 使用方法

```
adb root
```

```
adb shell setprop persist.vendor.avc.backtrace.enabled 1 //打开 avc backtrace 开关
```

```
adb shell echo all > /sys/fs/selinux/backtrace_filter //可以打印所有 avc 的 backtrace
```

```
adb shell echo ylog,slogmodem,rild > /sys/fs/selinux/backtrace_filter //可以打印  
ylog/slogmodem/rild 三个进程的 avc backtrace
```

这里最多可以输入 5 个进程的 comm 值，该值可以通过 `cat /proc/pid/comm` 来获取，各个 comm 名之间以逗号分割开。最终 `avc backtrace` 会打印到 `/data/tombstone` 文件里，当前 tombstone 文件的上限是 50 个，达到上限后，就不会再产生新的 tombstone 文件了，当 `avc` 较多时，删除较老的 tombstone 文件，或者设置 `tombstoned.max_tombstone_count` 属性来修改 tombstone 文件上限值。

Android10.0 堆栈会打印到 kernel log

Android9.0_19a 堆栈会打印到 `/data/tombstones`

注:通过 `echo` 写入 `/sys/fs/selinux/backtrace_filter` 的命令,手机重启会失效.如果涉及手机重启,需要手动修改 `/vendor/etc/init/avc_backtrace.rc` 文件并 push 到手机。

Unisoc Confidential For quectel_smart